MPTA-Repair: Simultaneous Multi-Property Clock Bound Repair for Networks of Timed Automata via Iterative Space Optimization

Guangtong Zhou, Run Chang, Ji Wu*
School of Computer Science and Engineering, Beihang University, Beijing, China
Email: zhougt@buaa.edu.cn, handsomeruncr@gmail.com, wuji@buaa.edu.cn

***Abstract*—Networks of timed automata (NTA) are widely used to model and verify real-time industrial systems, which are typically required to satisfy multiple properties simultaneously. When verification fails, repairing an NTA is challenging because a valid repair must modify erroneous clock constraints, satisfy all properties, and preserve the original functional behavior. We present MPTA-Repair, an automated repair framework for multi-property NTA. It uses an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidates, exploiting relations among properties, counterexamples, and repair variables to optimize search. Each candidate is validated by full-property regression verification and an admissibility check based on functional equivalence. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our mutation strategy. Specifically, it improves the success rate from 72% to 93% on benchmarks, and from 20% to 76% on an industrial dataset constructed in this work.**

***Index Terms*—networks of timed automata, automated model repair, MaxSMT, multi-property verification**

I. Introduction

Real-time systems are widely used in domains such as railway control, communication protocols, and aerospace scheduling [1]–[3]. Their correctness depends not only on whether they exhibit the intended discrete behavior, but also on whether relevant actions occur within the required time bounds. Timed automata (TAs) provide a formalism for modeling these two aspects by extending finite-state automata with real-valued clocks and timing constraints that govern both the occurrence of transitions and the time spent in locations [4], [5]. Systems composed of multiple interacting components are commonly represented as networks of timed automata (NTA). In practice, NTA are typically verified against sets of properties specifying the required system behavior, including safety and liveness requirements [6], [7].

Among the properties checked for an NTA, this work focuses on timed safety properties, which are fundamental and important for specifying bounded timing requirements, such as deadlines, limits on waiting times, and valid execution durations [7], [8]. Violations of such properties can be witnessed by finite timed executions, which provide concrete evidence for subsequent repair analysis [7], [9], [10]. In complex NTAs, multiple real-valued clocks advance simultaneously, and their constraints determine transition occurrence and residence time. An incorrect numerical clock bound may therefore affect executions across multiple components, causing an action to occur too early, too late, or for an invalid duration [2], [10], [11]. When such a violation is detected, the clock bounds must be repaired while preserving the intended functional behavior.

When a property violation is detected, real-time model checkers such as UPPAAL, Kronos, and opaal can return a timed counterexample, commonly called a timed diagnostic trace (TDT) [12]–[14]. A TDT describes a timed sequence of transitions and delays that leads the NTA from its initial state to a state in which the property is violated. It shows how the violation occurs, but does not explain which clock bounds caused it or how those bounds should be changed. The repair problem is more difficult when an NTA must satisfy multiple properties simultaneously: a candidate clock-bound change may resolve one TDT but leave other violations, make a previously satisfied property fail, or compromise functional behavior. Therefore, a candidate repair must be validated against the full property set and checked for admissibility based on functional equivalence [15].

Parameterization and constraint solving provide a general basis for repairing timing constraints in timed automata. A common strategy is to replace selected clock bounds with parameters and synthesize valuations for a specified property [16]. Knapik and Penczek encode bounded executions of parametric timed automata as SMT formulas for reachability properties [17]. TARTAR [10] encodes the timing constraints of a TDT in linear real arithmetic and uses MaxSMT to generate candidate repairs with minimal changes to clock bounds, followed by an admissibility check based on functional equivalence. Although these methods show that parameterization and constraint solving can support automated model repair, their repair or synthesis procedures are formulated for an individual property or TDT rather than a property set that must be satisfied simultaneously, and therefore do not provide an iterative repair strategy or search optimization for the multi-property setting.

To address this limitation, we present MPTA-Repair, an automated framework for simultaneous multi-property clock-bound repair of NTAs. The main contributions of this work are as follows. First, we formalize simultaneous multi-property

*Corresponding author.
The implementation and datasets are publicly available at https://github.com/zhougut/MPTA-Repair.

clock-bound repair for NTA models, together with the validity conditions of full-property satisfaction and preservation of the original functional behavior. Second, we propose an iterative, counterexample-guided MaxSMT framework that maintains a joint property-trace constraint pool and exploits relations among properties, counterexamples, and repair variables to optimize the repair search. Each candidate repair model undergoes full-property regression verification and an admissibility check; newly revealed TDTs are added to the pool for the next repair iteration. Third, we propose a mutation strategy for constructing faulty NTA models and evaluate MPTA-Repair against the TARTAR baseline [10] on benchmark models from UPPAAL examples [18], the XtaBenchmarkSuite [19], and industrial models constructed in this work, showing higher repair success rates in both settings.

The remainder of the paper is organized as follows. Section II introduces the notation and accepted-repair condition. Section III presents MPTA-Repair. Section IV reports the experimental evaluation. Section V concludes the paper.

II. Preliminaries

A. Networks of Timed Automata and Properties

The paper follows standard timed-automata semantics [5]. This section establishes the notation used for multi-property clock-bound repair.

Definition 1 (Timed automaton). A timed automaton (TA) is a tuple

$$T = (L, \ell_0, C, \Sigma, \Theta, I), \tag{1}$$

where L is a finite set of locations, ℓ_0 is the initial location, C is a finite set of clocks, Σ is a set of action labels, $\mathcal{B}(C)$ is the set of clock constraints over C, $\Theta \subseteq L \times \mathcal{B}(C) \times \Sigma \times 2^C \times L$ is a finite set of transitions, and $I : L \to \mathcal{B}(C)$ maps each location to a clock invariant. A clock constraint is a conjunction of atoms $x \sim b$, where $x \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and $b \in \mathbb{R}_{\geq 0}$. A transition $\theta = (\ell, g, a, R, \ell')$ moves from ℓ to ℓ' when guard g is satisfied, emits action a, and resets clocks in $R \subseteq C$. As mentioned above, this work focuses on repairing clock bounds. Specifically, a clock bound is the numerical constant b in an atomic clock constraint $x \sim b$ appearing in a transition guard or location invariant.

Definition 2 (NTA with property set). An NTA specification consists of a network of timed automata and the properties that the network must satisfy:

$$\begin{aligned} M &= (N, \Phi), \\ N &= T_1 \parallel \cdots \parallel T_k, \\ \Phi &= \{\varphi_1, \ldots, \varphi_m\}. \end{aligned} \tag{2}$$

The network N models a real-time system by composing interacting timed automata, with synchronization semantics supplied by the target modeling language, such as UPPAAL [12]. The property set Φ is part of the system specification: it states the timing and behavioral requirements that the NTA must satisfy. The notation $N \models \Phi$ means that N satisfies every property in Φ:

$$N \models \Phi \quad \Longleftrightarrow \quad \forall \varphi_i \in \Phi,\ N \models \varphi_i. \tag{3}$$

The repair problem considered here is restricted to timed safety properties, which require a state predicate to hold in every reachable state. In UPPAAL-style notation, such a property has the invariant form `A[]` p, where p is a state predicate composed of location predicates and clock constraints. For example, `A[](Controller.Waiting imply` $x \leq 10$) requires that clock x never exceed 10 while the controller is in the `Waiting` location.

B. Timed Diagnostic Traces

Definition 3 (Timed diagnostic trace). Given an NTA specification $M = (N, \Phi)$ and a property $\varphi \in \Phi$, a timed diagnostic trace (TDT) for φ is a feasible finite timed execution of N that reaches a state violating φ. When verification of φ fails, a real-time model checker can generate a TDT as a counterexample. In UPPAAL, such a counterexample consists of a sequence of states and transitions, together with clock constraints describing the associated timing information [12].

In MPTA-Repair, each TDT is treated as a property-trace constraint: the violated property identifies the requirement to be restored, and the trace identifies the timed behavior that makes the violation feasible. Section III uses such constraints from multiple properties in a joint solving process.

C. Functional Equivalence and Repair Acceptance

Following TARTAR [10], we adopt equality of untimed languages as the criterion for functional equivalence. Under this criterion, a timing repair should neither remove existing functional behavior of the original model nor introduce new functional behavior, even though it changes timing constraints to eliminate violations.

Definition 4 (Functional equivalence). For an NTA N, let $L_\mu(N)$ denote the untimed language obtained by abstracting from delays while preserving the discrete behavior feasible under the timing constraints. Two NTA models N_1 and N_2 are functionally equivalent if

$$L_\mu(N_1) = L_\mu(N_2). \tag{4}$$

Definition 5 (Accepted repair). Let $M_{bad} = (N_{bad}, \Phi)$ be a faulty NTA specification. A candidate repair $M_{rep} = (N_{rep}, \Phi)$ is obtained by changing only repairable clock bounds in guards and invariants. The candidate is accepted if and only if

$$N_{rep} \models \Phi \quad \wedge \quad L_\mu(N_{bad}) = L_\mu(N_{rep}). \tag{5}$$

The first conjunct corresponds to full-property regression verification: the model must satisfy the entire property set, not only the property that produced the current TDT. The second conjunct is the admissibility check based on untimed functional equivalence; it prevents repairs that make properties true by disabling required functional behavior.

III. Approach

A. Overview

MPTA-Repair takes as input $M = (N, \Phi)$, where N is an NTA and $\Phi = \{\varphi_1, \ldots, \varphi_m\}$ is a set of timed

safety properties, and modifies only repairable numerical clock bounds in transition guards and location invariants. It returns $M' = (N', \Phi)$ that satisfies the accepted-repair condition in Eq. (5); otherwise, it reports that no accepted repair was found within the resource budget.

The properties in Φ may not be independent, as different properties may be affected by the same clock bounds. Their TDTs may be identical, share part of an execution path, or depend on the same repairable bounds. A repair derived from one TDT may leave other violations unresolved or cause a satisfied property to fail. Treating TDTs separately ignores these relations and may lead to repeated, conflicting repairs.

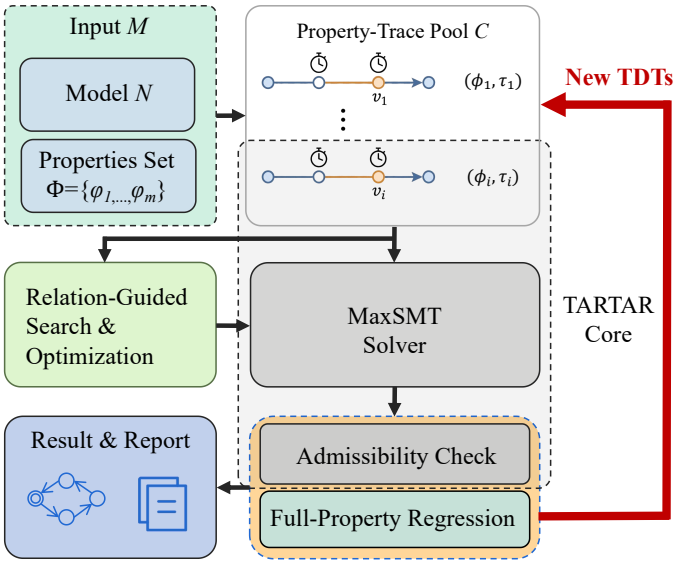

Fig. 1. Overview of the MPTA-Repair workflow.

Figure 1 summarizes the MPTA-Repair framework. Initial full-property verification either confirms that no repair is needed or initializes a joint property-trace constraint pool with TDTs. MPTA-Repair analyzes relations among properties, TDTs, and repairable bounds, removes redundant constraints, and solves constraints that share bounds jointly. The constraints are encoded as a MaxSMT problem, and the resulting assignment is applied to N to construct a candidate model. MPTA-Repair extends TARTAR's single-TDT MaxSMT repair procedure [10] by jointly processing related constraints and using newly generated TDTs in subsequent repair iterations.

Each candidate undergoes full-property regression verification and the admissibility check based on functional equivalence. If regression verification reveals new violations, the corresponding TDTs are added to the joint pool. Any assignment that fails either validation step is excluded from subsequent iterations. The process repeats until an accepted repair is found or the time budget is exhausted. Figure 2 details the relation analysis, MaxSMT solving, and validation stages.

B. Joint Trace Encoding

MPTA-Repair uses the clock-bound variation representation introduced in TARTAR [10] as the basis for its joint multi-property encoding and iterative repair procedure. Let S denote the set of repairable clock bounds. For each clock bound $s \in S$, the original numerical bound b_s in a transition guard or location invariant is associated with a variation variable δ_s. The repaired numerical bound b'_s is defined as:

$$b'_s = b_s + \delta_s \tag{6}$$

MPTA-Repair builds on the trace-encoding formulation of TARTAR and extends it to the multi-property setting [10]. Consider a TDT with n discrete transitions:

$$\begin{aligned}\tau = \ell_0 \xrightarrow{d_0,\theta_0} \ell_1 \cdots \\ \xrightarrow{d_{n-1},\theta_{n-1}} \ell_n\end{aligned} \tag{7}$$

Here, d_i is the delay before transition θ_i, where $\theta_i = (\ell_i, g_i, a_i, R_i, \ell_{i+1})$. Let $\rho = (b'_s)_{s\in S}$ denote the modified-bound vector, and let $\mathbf{z}_\tau = (d_0, \ldots, d_{n-1}, \nu_0, \ldots, \nu_n)$ collect the delay values and clock valuations along this execution, where ν_i is the clock valuation at location ℓ_i. Then the trace path is encoded as a conjunction of linear arithmetic constraints:

$$\begin{aligned}\mathrm{Path}(\tau,\rho,\mathbf{z}_\tau) = \mathit{Init}(\nu_0) \\ \wedge \bigwedge_{i=0}^{n-1} \Big(d_i \geq 0 \wedge \mathit{Inv}(\ell_i, \nu_i + d_i, \rho) \\ \wedge \mathit{Guard}(\theta_i, \nu_i + d_i, \rho) \\ \wedge \nu_{i+1} = \mathit{Reset}(\nu_i + d_i, R_i) \Big)\end{aligned} \tag{8}$$

Here, $\mathit{Init}(\nu_0)$ constrains the initial clock valuation, Inv evaluates the invariant $I(\ell_i)$, and Guard evaluates the guard g_i of transition θ_i under the given valuation and modified bounds. The reset equation $\nu_{i+1} = \mathit{Reset}(\nu_i + d_i, R_i)$ updates the valuation after transition θ_i by resetting clocks in R_i and preserving the others. The predicate $\mathit{Bad}_\varphi(\ell_n, \nu_n)$ characterizes the violation state of φ at the final location. To block this violating behavior while preserving executability of the discrete path skeleton, the single-trace repair constraint uses two disjoint copies $\mathbf{z}_\tau^+$ and $\mathbf{z}_\tau^-$ of the delay and clock-valuation variables:

$$\begin{aligned}\Psi^{\mathrm{repair}}_{\tau,\varphi}(\rho) = \exists \mathbf{z}_\tau^+ . \mathrm{Path}(\tau,\rho,\mathbf{z}_\tau^+) \\ \wedge \neg \exists \mathbf{z}_\tau^- . \Big(\mathrm{Path}(\tau,\rho,\mathbf{z}_\tau^-) \\ \wedge \mathit{Bad}_\varphi(\ell_n, \nu_n^-) \Big)\end{aligned} \tag{9}$$

The first part requires that the original discrete path skeleton still has at least one timed realization after repair. This preserves the observed discrete behavior: the repair must not remove the transition sequence simply to avoid the violation. The second part requires that no assignment of delays and clock valuations can make that same path violate φ.

We extend this single-trace formulation to a joint, multi-property incremental setting. At refinement round k, counterexamples from all currently violated properties are accumulated into a trace pool

$$\mathcal{T}_k = \{(\tau_j, \varphi_j)\}_{j=1}^{m_k} \tag{10}$$

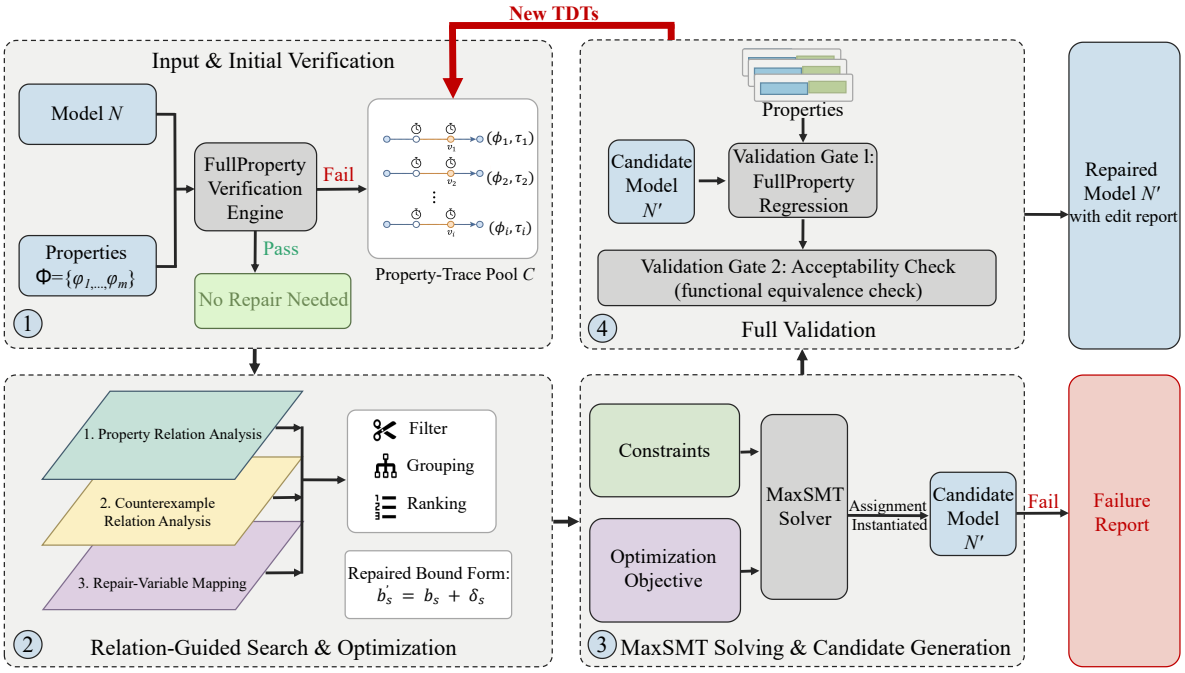

Fig. 2. Overview of the MPTA-Repair framework.

where $m_k = |\mathcal{T}_k|$ is the number of property-trace pairs in the pool at round k. MPTA-Repair assembles a joint MaxSMT problem with the constraint system defined as

$$\mathcal{F}_k(\rho) = Dom(\rho) \wedge Block_k(\rho) \wedge \bigwedge_{(\tau,\varphi)\in Select(\mathcal{T}_k)} \Psi^{\text{repair}}_{\tau,\varphi}(\rho) \tag{11}$$

Here, $Dom(\rho)$ enforces static interval consistency and domain limits, such as $b'_s \geq 0$; $Block_k(\rho)$ collects the blocking clauses accumulated before round k and is initially true; and $Select(\mathcal{T}_k)$ represents a reduced subset of traces obtained after relation-aware filtering. When an instantiated candidate NTA fails verification, the constraint system is updated incrementally. The framework expands the pool via

$$\mathcal{T}_{k+1} = \mathcal{T}_k \cup NewTDTs(N(\rho_k), \Phi) \tag{12}$$

where ρ_k is the modified-bound assignment produced in round k, $N(\rho_k)$ is the candidate network obtained by applying ρ_k to N, and $NewTDTs(N(\rho_k), \Phi)$ is the set of newly exposed property-trace pairs returned by full-property verification. The framework then updates the search-space block to exclude the failed assignment ρ_k over the set of repairable bounds S:

$$Block_{k+1}(\rho) = Block_k(\rho) \wedge \left(\bigvee_{s\in S} \delta_s \neq \delta_s^k \right) \tag{13}$$

where δ_s^k is the valuation assigned to variation variable δ_s in round k.

C. Relation-Guided Search Optimization

During refinement, accumulating violated properties, TDTs, and repairable bounds increases the size and complexity of the MaxSMT problem. To improve solving efficiency, MPTA-Repair uses relation-guided optimization to filter and group constraints and rank repairable bounds. These heuristics guide only the search; correctness is still guaranteed through full-property regression verification and an admissibility check.

MPTA-Repair analyzes relations at the property, counterexample, and repair-bound levels. Timed safety properties are grouped by the state predicates and clock constraints in their $(\texttt{A[]}, p)$ queries; identical queries are treated as duplicates, while tighter bounds are given higher priority among properties with the same structure. Each TDT is stored with the property it violates, and property-trace pairs sharing transitions or repair variables are grouped for joint solving. Each active property-trace constraint is then mapped to the repairable clock-bound occurrences in its encoding. Bounds occurring in newly added TDTs or associated with more TDTs and violated properties are prioritized. This ranking guides active-variable selection and candidate generation, whereas the number and magnitude of clock-bound changes are handled by the MaxSMT optimization objective.

To avoid repeated encoding across iterations, MPTA-Repair applies quantifier elimination (QE) to each property-trace constraint, eliminating its delay and trace-local clock variables to obtain a quantifier-free constraint over the repair variables. The resulting constraint is cached and reused in later rounds, so QE is required only for newly added property-trace pairs. All properties remain in Φ for full-property regression verification, and a constraint is removed from the active encoding only when it is a duplicate or logically implied by another active constraint. Within this reduced search space, MPTA-Repair minimizes deviation from the original NTA using the following

TABLE I
STATISTICS OF THE BENCHMARK AND INDUSTRIAL DATASETS.

Dataset	Orig.	Mut.	Avg. prop.	Avg. loc.	Avg. trans.
Benchmark	9	54	4.1	30.56	52.67
Industrial	5	105	7.3	57.80	91.80

Orig. and Mut. denote original and mutated models; Avg. prop., Avg. loc., and Avg. trans. denote average properties, locations, and transitions per model.

TABLE II
REPAIR RESULTS ACROSS BENCHMARK AND INDUSTRIAL DATASETS.

Dataset	Method	Mutated models	Repaired	Success rate	Avg. time
Benchmark	Iterative baseline	54	39	72%	60.61s
	MPTA-Repair	54	50	93%	15.46s
Industrial	Iterative baseline	105	21	20%	35.59s
	MPTA-Repair	105	80	76%	85.28s

lexicographic objective:

$$\operatorname*{lex\,min}_{\rho\in\Omega_k} \begin{pmatrix} \sum_{s\in S} \mathbf{1}\{\delta_s \neq 0\}, \\ \sum_{s\in S} |\delta_s| \end{pmatrix} \tag{14}$$

where Ω_k is the feasible assignment space defined by the current joint constraint system, and $\mathbf{1}\{\cdot\}$ is the indicator function. The first objective minimizes the number of modified clock-bound occurrences, and the second minimizes their total variation, keeping the repaired bounds close to those of the original NTA.

IV. EXPERIMENTAL EVALUATION

This section evaluates MPTA-Repair on benchmark and industrial datasets. The evaluation follows the clock-bound repair setting defined in Section III, where only numerical bounds in guards and invariants are repairable. We compare MPTA-Repair with an iterative baseline adapted from TARTAR [10].

A. Evaluation Setup and Research Questions

We evaluate MPTA-Repair on two datasets, summarized in Table I. The benchmark dataset includes nine benchmark models selected from UPPAAL examples [18] and the XtaBenchmarkSuite [19], such as Elevator and FDDI. The industrial dataset constructed in this work contains five closely coupled NTA case-study models: Gear Control System for automotive transmission [20], SAE-RoR for autonomous-vehicle road-junction rules [21], Train Controller Alarm for railway emergency timing [22], Train Gate Controller for shared-resource concurrency [4], and Pacemaker for medical pacing [11]. Compared with the benchmark models, these case studies involve more tightly coupled timing constraints across multiple properties.

Since real-world NTA models with labeled clock-bound faults are difficult to obtain, we synthesize faulty instances using mutation [23]. Some collected models contain few or no properties, so we supplement them with timed safety properties described or implied by their design documents or related papers. For each model, we seed modifications to all clock-constraint bounds by $\{-10, -1, +1, +0.1M, +M\}$, where M is the maximal bound compared against a clock in that model. We retain only mutants that are syntactically valid, violate at least one timed safety property, and preserve the original functional behavior. Each repair instance is executed with a 10-minute timeout for both MPTA-Repair and the baseline.

The iterative baseline adapted from TARTAR [10] repairs one violated property and its TDT at a time. If validation exposes a new TDT, it starts another repair round rather than solving all TDTs jointly. The evaluation is organized around the following research questions:

RQ1. What are the main characteristics of the benchmark and industrial repair instances?

RQ2. How does MPTA-Repair compare with the iterative TARTAR baseline in terms of accepted-repair rate?

RQ3. How does iterative space optimization affect repair efficiency and effectiveness?

RQ4. What practical lessons emerge from applying MPTA-Repair to industrial cases, and what explains its advantages over the iterative TARTAR baseline and its remaining failures?

B. Results by Research Question

RQ1: Dataset and mutation characteristics. Table I shows that the benchmark dataset contains nine original models and 54 retained mutants, while the industrial dataset contains five original models and 105 retained mutants. Although it has fewer original models, the industrial dataset is larger per model: 57.80 locations, 91.80 transitions, and 7.3 properties on average, compared with 30.56 locations, 52.67 transitions, and 4.1 properties for the benchmark dataset. The repair instances therefore cover both smaller benchmark models and larger industrial models with broader property sets.

RQ2: Accepted-repair rate. Table II shows that MPTA-Repair achieves higher accepted-repair rates than the iterative baseline on both datasets. On the benchmark dataset, it improves the success rate from 72% to 93% (39 to 50 repairs). On the industrial dataset, the improvement is larger, from 20% to 76% (21 to 80 repairs). The average time should be read together with these success rates. MPTA-Repair is faster on the benchmark dataset, but takes longer on the industrial dataset because it continues searching for joint repairs over multiple properties and TDTs. The baseline often stops earlier after failing to find a valid candidate, which lowers its average time but also yields far fewer accepted repairs. This supports the use of joint property-trace solving when industrial timing constraints are tightly coupled.

RQ3: Effect of iterative space optimization. Table III compares MPTA-Repair with and without search optimization. On the benchmark dataset, the average iteration count is already low (1.27 without optimization and 1.26 with optimization), so the time reduction is small, from 15.74s to 15.46s. On the industrial dataset, where coupled violations often require more refinement rounds, optimization reduces the average iteration

TABLE III
AVERAGE REPAIR TIME AND ITERATIONS WITH AND WITHOUT SEARCH OPTIMIZATION.

Dataset	Method	Avg. time	Avg. iter.
Benchmark	Non-optimized	15.74s	1.27
	Optimized	15.46s	1.26
Industrial	Non-optimized	231.45s	2.20
	Optimized	85.28s	1.40

count from 2.20 to 1.40 and the average time from 231.45s to 85.28s. Since the optimization does not change the repair space or accepted-repair condition, it affects efficiency rather than repair effectiveness.

RQ4: Practical lessons and failure analysis. The industrial cases show that accepted repairs require both global property checking and functional admissibility. A candidate that blocks one TDT may still violate another property, while a bound edit may satisfy timing properties only by removing intended behavior. MPTA-Repair rejects such candidates through full-property regression and functional-equivalence checking. This also explains its advantage over the iterative TARTAR baseline: local single-TDT repairs often fail on coupled industrial models, whereas the joint pool captures shared timing constraints. Remaining failures mainly occur when no admissible candidate is found within the time budget, as in Pacemaker, where dense timing interactions enlarge the clock-bound repair space.

V. CONCLUSION

This paper presented MPTA-Repair, an automated framework for simultaneous multi-property clock-bound repair of NTAs. It uses an iterative, counterexample-guided MaxSMT procedure that maintains a joint property-trace constraint pool and exploits relations among properties, counterexamples, and repair variables to optimize the repair search. Each candidate undergoes full-property regression verification and the admissibility check based on functional equivalence [10]; any newly revealed TDT is added to the pool for the next repair iteration.

The experimental results show that MPTA-Repair achieves higher repair success than the TARTAR baseline, especially on industrial models with coupled timing constraints. The remaining failures mainly occur when such interactions enlarge the clock-bound repair space and prevent an admissible candidate from being found within the time budget.

Future work will improve repair efficiency by checking candidate admissibility earlier, allowing functionally invalid candidates to be discarded before full validation. We will also extend the repair space beyond numerical clock bounds to synchronization labels and discrete transition structures, together with new admissibility conditions for preserving the intended functional behavior.

REFERENCES

[1] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer: Applying formal methods to railway standard interfaces," in *Proceedings of Formal Methods for Industrial Critical Systems*, ser. Lecture Notes in Computer Science, vol. 12863. Springer, 2021, pp. 174–190.

[2] H. Lönn and P. Pettersson, "Formal verification of a TDMA protocol start-up mechanism," in *Proceedings of the 1997 IEEE Pacific Rim International Symposium on Fault-Tolerant Systems*, 1997, pp. 235–242.

[3] M. Mikučionis, K. G. Larsen, J. I. Rasmussen, B. Nielsen, A. Skou, S. U. Palm, J. S. Pedersen, and P. Hougaard, "Schedulability analysis using Uppaal: Herschel-planck case study," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, ser. Lecture Notes in Computer Science, vol. 6416. Springer, 2010, pp. 175–190.

[4] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[5] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[6] E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., *Handbook of Model Checking*. Springer, 2018.

[7] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[8] S. Konrad and B. H. C. Cheng, "Real-time specification patterns," in *Proceedings of the 27th International Conference on Software Engineering*. ACM, 2005, pp. 372–381.

[9] O. Kupferman and M. Y. Vardi, "Model checking of safety properties," *Formal Methods in System Design*, vol. 19, no. 3, pp. 291–314, 2001.

[10] M. Kölbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," in *International Conference on Computer Aided Verification*. Springer, 2019, pp. 79–96.

[11] Z. Jiang, M. Pajic, A. Connolly, S. Dixit, and R. Mangharam, "Real-time heart model for implantable cardiac device validation and verification," in *Proceedings of the Euromicro Conference on Real-Time Systems*. IEEE Computer Society, 2010, pp. 239–248.

[12] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[13] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[14] A. E. Dalsgaard, R. R. Hansen, K. Y. Jørgensen, K. G. Larsen, M. C. Olesen, P. Olsen, and J. Srba, "opaal: A lattice model checker," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 6617. Springer, 2011, pp. 487–493.

[15] M. Kölbl, S. Leue, and T. Wies, "Automated repair for timed systems," *Formal Methods in System Design*, vol. 59, pp. 136–169, 2021.

[16] É. André, "What's decidable about parametric timed automata?" *International Journal on Software Tools for Technology Transfer*, vol. 21, no. 2, pp. 203–219, 2019.

[17] M. Knapik and W. Penczek, "SMT-based parameter synthesis for parametric timed automata," in *Challenging Problems and Solutions in Intelligent Systems*, ser. Studies in Computational Intelligence, G. de Tré, P. Grzegorzewski, J. Kacprzyk, J. W. Owsiński, W. Penczek, and S. Zadrożny, Eds. Cham: Springer, 2016, vol. 634, pp. 3–21.

[18] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.

[19] R. Farkas and G. Bergmann, "Towards reliable benchmarks of timed automata," in *Proceedings of the 25th PhD Mini-Symposium*, 2018, pp. 20–23.

[20] M. Lindahl, P. Pettersson, and W. Yi, "Formal design and analysis of a gear controller," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, ser. Lecture Notes in Computer Science, vol. 1384. Springer, 1998, pp. 281–297.

[21] G. V. Alves, L. A. Dennis, and M. Fisher, "A double-level model checking approach for an agent-based autonomous vehicle and road junction regulations," *Journal of Sensor and Actuator Networks*, vol. 10, no. 3, p. 41, 2021.

[22] A. González, M. Cristiá, and C. Luna, "Verification of quantitative temporal properties in RealTime-DEVS," *SIMULATION: Transactions of The Society for Modeling and Simulation International*, vol. 101, no. 10, pp. 1057–1076, 2025.

[23] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.